Impact of Deep Compression Techniques on Phase-Specific Accuracy in TCN for Real-Time Gait Detection

Anonymous Author(s)
Affiliation withheld for double-anonymous review

***Abstract*—Wearable-based gait phase detection is a critical enabler for rehabilitation monitoring, fall-risk assessment, and assistive robotics, yet deploying deep learning models on resource-constrained microcontrollers remains challenging. This study investigates the impact of deep compression techniques—post-training INT8 quantization and structured channel pruning—on the phase-specific accuracy of a Standard Temporal Convolutional Network (TCN) for real-time gait phase classification on the ESP32-C3 microcontroller. Using shank-mounted IMU data from the NONAN GaitPrint dataset, we evaluate all models under subject-wise stratified 5-fold cross-validation (K=5, three seeds), quantify phase-specific accuracy degradation (LR, LS, PSw, Sw) under INT8 quantization and structured pruning applied to TCN, and map the efficiency–accuracy trade-off via a Pareto front of macro F1-score versus on-device model size (stride-1 test evaluation; per-subject blocked macro F1 reported alongside window-level metrics). Under the primary K=5 × 3 baseline protocol, TCN achieved the highest mean macro F1 among compared architectures ($90.87\% \pm 1.47\%$), significantly outperforming CNN1D ($+0.53$ pp, Wilcoxon p=0.004) while remaining statistically equivalent to Transformer (p=0.80) and LSTM+GRU (p=0.17). K-fold compression (N=15 matched fold×seed pairs per config) shows that INT8 quantization is effectively lossless *under a PyTorch weight-only quantize-dequantize (QDQ) proxy* ($90.88\% \pm 1.47\%$ vs. FP32; p=0.30)—but evaluating the *actual deployed* full-integer TFLite model against the complete test set, matching the primary N=15 protocol (5 folds × 3 seeds), reveals a far larger loss (mean 11.15 ± 9.51 pp, $p=6.1 \times 10^{-5}$ vs. FP32), with one fold collapsing to as low as 56.98% (-35.13 pp) under every seed tested— implicating the held-out subject cohort, not calibration randomness, and disproportionately affecting the same transition phases found sensitive to pruning: the QDQ proxy is not a reliable stand-in for deployed accuracy. Structured pruning, which involves no quantization and carries no such proxy gap, reduces macro F1 by 1.47 pp at 50% channel retention with a 5-epoch fine-tune budget ($p<0.001$); extending fine-tuning to 50 epochs recovers 0.94 pp ($90.34\% \pm 1.11\%$, p=0.015 vs. FP32). PSw F1—the most compression-sensitive transition phase—drops by up to 3.75 pp under pure Prune25 pruning; LR F1 falls by up to 22.1 pp on average under real deployed INT8. INT8+Prune50's deployed footprint (13.4 KB) shows a 13.10 ± 6.59 pp real accuracy cost across the same N=15 protocol, not the QDQ proxy's claimed 1.54 pp. On ESP32-C3, INT8+Prune50 achieves 75.7 ms mean latency per 0.5 s window—3.7× faster than full-width INT8 (279.8 ms)—whereas FP32 inference exceeds the window duration (856 ms); accuracy claims for any quantized on-device config should be validated on the exact deployed artifact.**

***Index Terms*—TCN, deep compression, quantization, pruning, gait phase detection, ESP32-C3, edge AI, wearable sensors**

I. Introduction

Accurate detection of gait phases—such as loading response, mid-stance, pre-swing, and swing—is fundamental to applications in clinical rehabilitation, fall prevention, prosthetic control, and sports biomechanics. Recent advances in deep learning, particularly recurrent and convolutional architectures, have substantially improved gait phase classification accuracy using wearable inertial measurement unit (IMU) sensors. However, these models are typically designed and validated on desktop or server-class hardware, and their large parameter counts and floating-point computations make them impractical for direct deployment on low-power, low-memory microcontrollers used in real-world wearable devices.

Temporal Convolutional Networks (TCNs), which leverage dilated convolutions to capture long-range temporal dependencies with fewer parameters than recurrent architectures, offer a promising middle ground between accuracy and efficiency and can be further compressed via quantization and pruning to meet the strict memory and latency budgets of microcontroller-class hardware such as the ESP32-C3. While prior work has explored model compression broadly for time-series classification, little attention has been paid to how compression affects accuracy unevenly across gait phases. Transition phases such as Loading Response (LR) and Pre-Swing (PSw) are inherently more ambiguous due to overlapping biomechanical signal characteristics, and we hypothesize these are more vulnerable to information loss from aggressive quantization and pruning than steady-state phases such as Swing (Sw).

This study addresses this gap by:

1) Selecting and training a Standard TCN backbone for gait phase classification using shank IMU data from NONAN GaitPrint, with matched-capacity FP32 baselines for context;
2) Systematically applying INT8 quantization and structured pruning, quantifying their impact on phase-specific accuracy and F1-score rather than overall accuracy alone;
3) Deploying the compressed models on ESP32-C3 hardware and measuring on-device latency and SRAM headroom.

II. Literature Review

A. Gait Phase Detection Using Wearable IMU Sensors

Gait phase detection using body-worn IMU sensors has been extensively studied, with shank- and foot-mounted sensors commonly used due to their high sensitivity to phase transitions. Classical approaches (threshold-based, hidden Markov models) have largely been superseded by deep learning methods—1D CNNs, LSTM, hybrid LSTM+GRU, and, more recently, Transformer encoders—which learn temporal dependencies directly from raw or lightly processed signals. Using the same NONAN GaitPrint dataset and shank-mounted six-axis IMU configuration, Choi and Choi [1] directly compared a 1D CNN, a hybrid LSTM+GRU model, and a Transformer for four-class gait phase classification, reporting comparable performance (macro F1 ≈91–93%) with the Transformer attaining the highest score. The present study replicates those three architectures, plus a standard TCN, as FP32 baselines under an identical protocol (Section III), but targets a distinct research question left open by prior work: how INT8 quantization and structured pruning affect *phase-specific* accuracy, and how the resulting models perform when deployed on microcontroller-class hardware.

B. Temporal Convolutional Networks (TCNs) for Time-Series Classification

TCNs use causal, dilated convolutions to achieve large receptive fields with a fraction of the parameters required by recurrent architectures, while supporting parallelized computation [2], making them attractive for edge deployment where both parameter count and latency are critical. All architectures here share a common hidden width (32 channels) and 0.5 s causal windows, yielding 10–26K-parameter models sized for ESP32-C3 rather than server-scale capacity.

C. Model Compression Techniques: Quantization and Pruning

Post-training quantization reduces numerical precision (e.g., FP32→INT8), reducing model size and enabling faster, more energy-efficient inference on integer-optimized hardware [3]. Structured pruning removes entire channels or filters, yielding actual speedups on commodity hardware (unlike unstructured pruning, which needs sparse-matrix support) [4]. Both are widely studied for image and audio models, but their differential effect across distinct temporal phases of a periodic signal—such as gait phases—and the gap between simulated and deployed quantization accuracy, remain comparatively underexplored; this study addresses both directly.

D. Edge AI Deployment on Microcontrollers (ESP32-C3)

TensorFlow Lite Micro (TFLM) enables compact neural networks to run directly on microcontroller hardware with kilobyte-scale memory budgets [5]. The ESP32-C3 is a single-core RISC-V microcontroller (160 MHz, ≈400 KB SRAM [6]), a low-cost TinyML target that lacks the vector acceleration of higher-end Espressif parts, motivating its use here to stress-test compressed models under real hardware constraints.

III. Methodology

A. Dataset and Preprocessing

- **Dataset:** NONAN GaitPrint [7], using shank-mounted IMU channels (accelerometer and gyroscope axes).
- **Sampling rate:** Original 200 Hz signal downsampled to 100 Hz for all reported experiments, balancing temporal resolution against on-device resource constraints.
- **Labeling:** Gait cycle phases labeled as Loading Response (LR), Loading Stance (LS), Pre-Swing (PSw), and Swing (Sw).
- **Data split:** Subject-wise stratified 5-fold CV (K=5) with three seeds; validation subjects rotate per fold (resampled from the non-test pool) to avoid repeated early-stopping bias; no subject leakage across train/val/test.
- **Normalization:** Per-channel z-score normalization computed on the training set only.
- **Windowing:** Causal 0.5 s windows with train stride 5 and validation/test stride 1 (test windows overlap ≈98%; per-subject blocked macro F1 reported alongside window-level metrics).
- **Training:** Adam optimizer ($\mathrm{lr} = 10^{-3}$), batch size 512, up to 50 epochs; best checkpoint selected by validation macro F1.
- **Fair comparison design:** All architectures use hidden width 32, four-class output, identical windowing/optimization/CV splits, and capped capacity (∼10–26K params, ∼40–100 KB FP32) rather than maximizing accuracy with large networks.
- **Compression fine-tuning:** Pruning uses *training subjects only* (fixed epoch budget, no validation access) to avoid double-dipping on the FP32 checkpoint-selection validation set; runtime assertions verify zero subject overlap for every fold.
- **Evaluation:** Table I reports mean ± std macro F1 over K=5 folds×3 seeds (N=15 runs) with paired Wilcoxon tests; compression results (Table II) and the fine-tune ablation (Table IV) use the same protocol. Figs. 2 and 3 illustrate one fold/seed (0/42); Table V reports ESP32-C3 latency for four deployable configs from that same checkpoint.

Phase support is highly imbalanced (LS and Sw together account for $(1{,}274{,}499 + 1{,}016{,}857)/2{,}583{,}680 \approx 88.7\%$ of test windows, vs. only 11.3% for the transition phases LR and PSw combined). To prevent the majority phases from dominating optimization, all models—including the four FP32 baselines and every compression fine-tuning stage—are trained with a class-balanced weighted cross-entropy loss:

$$\mathcal{L} = -\sum_{c=1}^{4} \alpha_c \, y_c \log \hat{y}_c, \qquad \alpha_c = \frac{N}{4 \cdot N_c}, \tag{1}$$

where N is the number of training windows, N_c is the number of windows labeled with phase c, y_c is the one-hot ground-truth indicator, and $\hat{y}_c$ is the softmax output probability for phase c. This directly motivates reporting *macro*, rather than micro-averaged, metrics throughout this paper.

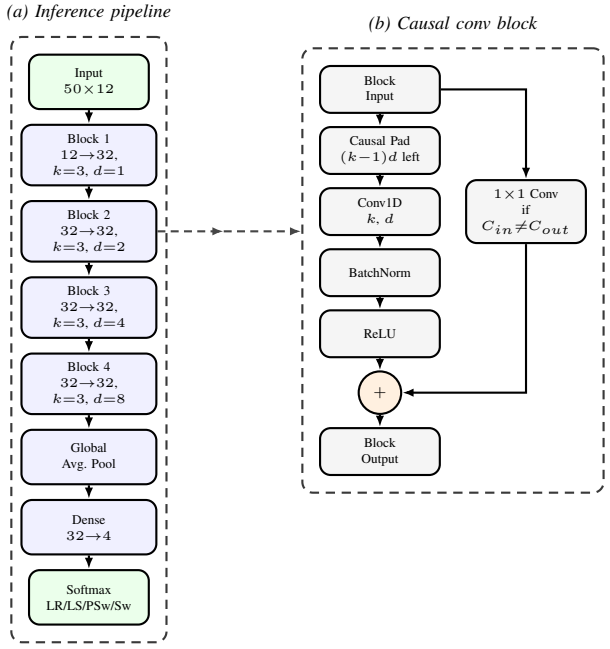

Fig. 1. Standard TCN architecture: (a) end-to-end pipeline from a 50×12 causal window to a 4-class softmax, and (b) internal structure of each causal convolutional block (left-padding, dilated Conv1D, batch norm, ReLU, and residual connection).

B. TCN Architecture

We adopt a Standard TCN [2] as the deployment backbone. The model consists of four dilated causal convolutional blocks (dilation rates 1, 2, 4, and 8) with residual connections, batch normalization, and a lightweight classification head (global average pooling followed by a dense softmax layer over the four phase classes). Each block computes a dilated causal convolution (kernel size k=3, dilation d, ReLU) so that each output depends only on present and past inputs. Stacking four such blocks yields a receptive field of 31 samples (0.31 s at 100 Hz), comfortably within the 0.5 s (50-sample) input window while keeping the parameter count near 11K. The model consumes causal 0.5-second windows (50 samples at 100 Hz). Channel width is deliberately kept narrow (32 hidden channels) to retain a microcontroller-deployable footprint ($\sim$45 KB FP32). Fig. 1 illustrates the overall data flow and the internal structure of each causal convolutional block.

C. Baseline Architectures for Context

For context, three FP32 baselines (CNN1D, LSTM+GRU, Transformer) were retrained under the same hidden width, windowing, and cross-validation protocol (Table I).

D. Deep Compression Protocol

Post-training quantization (INT8). We apply symmetric 8-bit quantization separately to each convolutional and fully connected weight layer. For every layer, a single scale is set from that layer's largest absolute weight; each weight is rounded to the nearest integer in $[-127, 127]$, then mapped back to floating point for inference. This quantize–dequantize (QDQ) proxy is used for PyTorch-side test-set evaluation (Section IV). For ESP32 deployment, models are exported as full-integer TensorFlow Lite graphs using TFLite's built-in post-training quantization.

Structured channel pruning. The TCN stem uses 32 hidden channels. Channels are ranked by summed absolute weight magnitude across all stem convolutional blocks; the top-k channels are retained and removed elsewhere in the stem and classification head. The pruned model is fine-tuned on *training subjects only* (Adam, learning rate 10^{-4}, class-balanced loss) before re-evaluation. Primary k-fold configs sweep 25%, 50%, and 75% channel retention (Prune25/50/75; Prune25 *retains* 25% and is thus the most aggressive setting, Prune75 the mildest) with a default 5-epoch fine-tune budget; Table IV reports the same protocol at N=15.

Extended compression grid (k-fold). Beyond FP32/INT8 and Prune25/50/75, combined INT8+Prune50 populates the Pareto front (Fig. 2). INT4 is excluded because full-integer INT4 TFLite is not supported on the ESP32-C3 deployment target. Fine-tune schedule ablation on Prune50 (5/15/50 epochs) isolates whether accuracy loss under pruning reflects insufficient recovery training rather than an inherent pruning limit (Table IV).

Compression variants comprise INT8 post-training quantization (PyTorch QDQ proxy for test metrics; full-integer TFLite PTQ for ESP32), structured 25/50/75% channel pruning with train-only fine-tuning, and combined INT8+Prune50.

E. Hardware Deployment (ESP32-C3)

After PyTorch compression evaluation, selected models are converted to TensorFlow Lite (.tflite) format and deployed via TensorFlow Lite Micro on an ESP32-C3 development board (160 MHz, Arduino IDE, Chirale_TensorFlowLite). On-device inference latency is measured per 0.5-second input window over 1000 timed trials (20 warmup invocations), alongside minimum free heap during benchmarking and a fixed 120 KB tensor arena.

F. Evaluation Metrics

Macro F1 is the unweighted mean of the four per-phase F1 scores. Unlike accuracy or a support-weighted average, this gives Loading Response (LR) and Pre-Swing (PSw) equal weight to the majority phases despite their $\sim 6\%$ and $\sim 5\%$ test-set support, respectively.

- Phase-specific F1 and accuracy for LR, LS, PSw, and Sw.
- Pareto analysis: macro F1 vs. model size (KB) per compression config.
- On-device latency (ms) and minimum free heap (KB) on ESP32-C3.

IV. Results

A. Baseline Comparison and Model Complexity

Table I compares TCN against three FP32 baselines under the primary K=5 × 3 protocol (mean ± std over 15 runs). Macro F1 scores cluster tightly (90.34–90.87%); TCN attains the highest mean macro F1 ($90.87\% \pm 1.47\%$). Paired

TABLE I
BASELINE FP32 COMPARISON (K=5 × 3 SEEDS)

Model	Params	KB	Macro F1 (%)	Phase F1 (%)			
				LR	LS	PSw	Sw
TCN	11,560	45.2	**90.87 ± 1.47**	**85.81**	**97.78**	82.72	97.18
Transformer	25,956	101.4	90.83 ± 0.99	85.37	97.73	**82.95**	**97.27**
LSTM+GRU	12,356	48.3	90.59 ± 1.19	84.95	97.70	82.52	97.22
CNN1D	10,727	41.9	90.34 ± 1.21	84.32	97.52	82.54	97.00

Mean ± std macro F1 (N=15); phase F1 reported as means. Hidden width 32 for all models.

TABLE II
TCN K-FOLD COMPRESSION (MACRO F1 %, MEAN ± STD; N=15 PER CONFIG)

Config	Macro F1	PSw F1	Wilcoxon p vs. FP32
FP32	90.87 ± 1.47	82.72 ± 2.54	—
INT8	90.88 ± 1.47	82.75 ± 2.54	0.303
Prune25	87.62 ± 1.45	78.97 ± 1.28	6.10e-05
Prune50	89.40 ± 1.43	81.13 ± 1.97	6.10e-05
Prune75	90.17 ± 1.30	81.90 ± 2.18	6.10e-05
INT8+Prune50	89.33 ± 1.36	80.93 ± 2.00	1.22e-04

Paired Wilcoxon signed-rank tests on matched fold×seed pairs (N=15) vs. FP32. All configs complete (120/120 jobs). p=6.10e−05 is the N=15 Wilcoxon floor (see Limitations). Config names denote % channels retained; Prune25 is most aggressive, Prune75 mildest.

Wilcoxon tests on matched fold×seed pairs show TCN statistically equivalent to Transformer (Δ=+0.04 pp, p=0.80) and LSTM+GRU (+0.28 pp, p=0.17), but significantly higher than CNN1D (+0.53 pp, p=0.004).

B. TCN Compression Under K-Fold Cross-Validation

Table II reports the primary compression results aggregated over N=15 matched fold×seed pairs, with paired Wilcoxon signed-rank tests against the FP32 reference (same checkpoints, no retraining).

INT8 quantization is statistically indistinguishable from FP32 under the PyTorch QDQ proxy—but this proxy substantially overstates deployed accuracy. INT8 attains 90.88% ± 1.47% macro F1 (+0.01 pp vs. FP32; Wilcoxon p=0.30) with an estimated payload of ∼13.0 KB (∼3.4× smaller than FP32) under the PyTorch weight-only quantize-dequantize (QDQ) proxy used throughout Table II. This proxy quantizes only weights; it does not model activation quantization noise. We therefore evaluated the *actual deployed* full-integer TFLite artifacts (Section IV-E) against the complete stride-1 test set (not a subsample) using the TFLite interpreter, across the same primary protocol as the rest of this paper (N=15: 5 folds × 3 seeds; Table III).

The real deployed INT8 model loses 11.15%±9.51% macro F1 on average (p=6.1 × 10^{-5} vs. FP32)—not the +0.01 pp the proxy predicts. Critically, fold 4 is the worst fold under *every* seed for both quantized configs (Table III), implicating the specific held-out subject cohort rather than calibration randomness; a spot-check re-calibrating fold 4/seed 42 with 4× more windows improved but did not close the gap (70.44% → 75.28%), so calibration quality compounds, but does not solely cause, a more fundamental sensitivity. Per-phase (mean over N=15), INT8 degrades LR far more (85.81% → 63.74%, −22.1 pp) than Sw (97.18% → 89.77%, −7.4 pp)—the same transition-phase vulnerability identified under pruning also appears, more severely, under real quantization noise. INT8+Prune50 shows the identical pattern (Table III). **The QDQ proxy is not a reliable stand-in for deployed INT8 accuracy**; any accuracy claim for a quantized model should be validated on the exact exported artifact.

TABLE III
REAL DEPLOYED TFLITE VS. PYTORCH ACCURACY (MACRO F1 %, TCN, N=15)

Config	Real TFLite	Δ vs. FP32	Δ vs. proxy	Wilcoxon p
INT8	79.72 ± 9.08	−11.15 ± 9.51	−11.16	6.10e-05
INT8+Prune50	77.78 ± 6.73	−13.10 ± 6.59	−11.56	6.10e-05

Real TFLite: full test set (stride-1) via interpreter on the exact exported artifact, all 5 folds × 3 seeds; FP32/proxy means in Table II. Worst case: INT8 fold 4/seed 123 (56.98%, −35.13 pp); INT8+Prune50 fold 4/seed 42 (65.87%, −25.47 pp). Fold 4 is worst under every seed for both configs (63.5%/67.6% mean vs. 82–87% for folds 0–3), implicating the held-out cohort over calibration randomness.

Pruning severity trades size for accuracy in a monotonic pattern. Unlike INT8, pruning involves no quantization and carries no proxy–deployment gap. Prune25 (∼4.3 KB estimated payload) reduces macro F1 by 3.25 pp (87.62% ± 1.45%, p<0.001), with the largest phase-specific loss in LR (−6.98 pp) and PSw (−3.75 pp). Prune50 (∼13.1 KB estimated payload) costs 1.47 pp (89.40% ± 1.43%, p<0.001). Prune75 (∼26.4 KB estimated payload) costs 0.70 pp (90.17% ± 1.30%, p<0.001). All pruned configs remain significantly below FP32 at α=0.05, but the effect size shrinks as more channels are retained.

Fig. 2 shows the *estimated-payload* Pareto front built from QDQ-proxy accuracy (Table V has measured sizes); quantized configs need re-validation on the deployed artifact. Among configs with no quantization gap, Prune75 has the best accuracy (∼26.4 KB, −0.70 pp) but measured on-device latency (510.8 ms) exceeds the 500 ms budget—accuracy alone does not predict deployability. Prune50 is the most defensible *real-time* choice with no proxy gap (−1.47 pp, confirmed 253 ms). INT8 variants are smaller/faster still but need per-checkpoint validation given the accuracy variance above.

C. Fine-Tuning Schedule Ablation

Table IV isolates whether accuracy loss under Prune50 reflects an insufficient post-prune recovery budget rather than an inherent pruning limit. With the default 5-epoch fine-tune, Prune50 macro F1 is 1.47 pp below FP32 (p<0.001). Extending fine-tuning to 15 epochs recovers 0.43 pp (89.83% ± 1.54%, p=6.1×10^{-4} vs. FP32), and 50 epochs recovers 0.94 pp (90.34% ± 1.11%, p=0.015 vs. FP32). PSw F1 improves from 81.13% ± 1.97% (5 ep) to 82.23% ± 1.82% (50 ep), approaching the FP32 baseline (82.72% ± 2.54%). Residual macro F1 gap at 50 epochs (−0.53 pp, still significant at α=0.05) indicates that structured 50% channel removal

TABLE IV
PRUNE50 FINE-TUNING SCHEDULE ABLATION (K-FOLD; N=15; WILCOXON VS. FP32)

Config	FT ep.	Macro F1	Δ vs. FP32	PSw F1	p
Prune50	5	89.40 ± 1.43	-1.47	81.13 ± 1.97	6.10e-05
Prune50 (15 ep)	15	89.83 ± 1.54	-1.04	81.29 ± 2.79	6.10e-04
Prune50 (50 ep)	50	90.34 ± 1.11	-0.53	82.23 ± 1.82	0.015

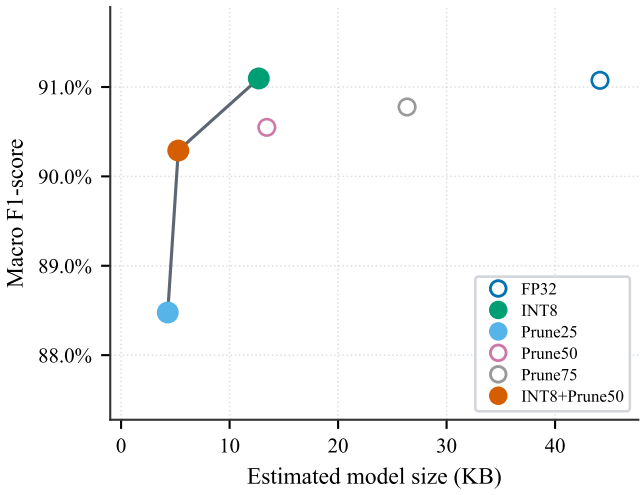

Fig. 2. Illustrative Pareto front (fold 0, seed 42). Horizontal axis is *estimated* payload, not measured TFLite size (Table V); k-fold aggregates in Table II. Frontier: Prune25 → INT8+Prune50 → INT8; Prune50/Prune75/FP32 are dominated here. Prune25 *retains* only 25% of channels (most aggressive); Prune75 is mildest.

retains a small but reproducible information loss beyond what longer fine-tuning can recover.

D. Compression Trade-offs and Phase Degradation

Figs. 2 and 3 qualitatively illustrate the accuracy–size trade-off and phase-specific degradation on a single fold×seed pair (fold 0, seed 42) under the QDQ proxy, consistent with the k-fold aggregates in Tables II and IV *as proxy accuracy*—but Table III shows the real deployed INT8 artifact is not near-lossless, so this Pareto view is an upper bound on quantized-config accuracy.

E. On-Device Performance on ESP32-C3

Table V reports on-device latency for five TFLite exports derived from the fold 0, seed 42 compression checkpoints. Benchmarks replay one fixed normalized $(50, 12)$ window in a tight loop (N=1000 trials after 20 warmup invocations) without live IMU or BLE I/O.

The 500 ms budget assumes one inference per non-overlapping window (hop = window), *not* the stride-1 (10 ms hop) offline protocol (Section III), which would need sub-10 ms inference (no config achieves this). Three of five configs meet the 500 ms budget: INT8+Prune50 (75.7 ms), Prune50 (253 ms), and full-width INT8 (279.8 ms); FP32 (856 ms) and, notably, **Prune75 (**510.8 **ms) both exceed it** despite Prune75

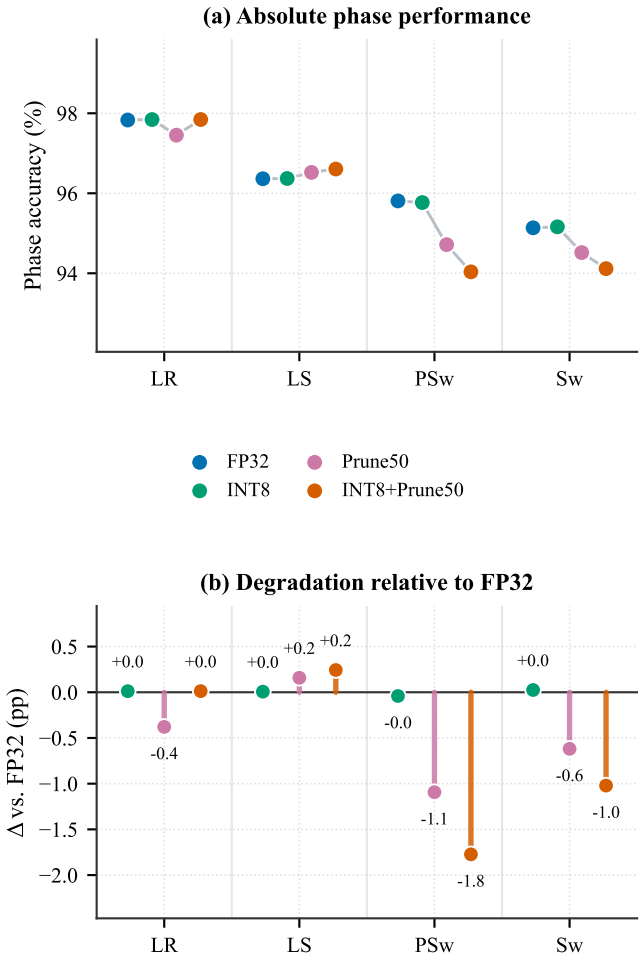

Fig. 3. Phase-specific test *accuracy* under compression (fold 0, seed 42). Panel (b) shows that phase *accuracy* can remain nearly flat while phase F1 falls under compression (INT8+Prune50: LR accuracy +0.01 pp vs. LR F1 −1.32 pp; PSw F1 −1.32 pp). Structured pruning also reduces PSw/Sw accuracy (up to −1.8 pp), motivating macro and phase F1 over accuracy alone.

TABLE V
ON-DEVICE INFERENCE ON ESP32-C3 (STANDARDTCN, FOLD 0, SEED 42)

Config	Mean (ms)	p95 (ms)	TFLite (KB)	Heap (KB)
FP32	856.2	856.2	52.1	116.3
Prune75	510.8	510.8	34.5	156.3
INT8	279.8	279.8	21.3	156.3
Prune50	253.0	253.0	21.6	156.3
INT8+Prune50	75.7	75.7	13.4	156.3

ESP32-C3 @ 160 MHz; TFLite Micro. Latency/0.5 s window (N=1000, 20 warmup). Heap = min. free dynamic heap; arena 120 KB (all but FP32) or 160 KB (FP32); weights in flash. Prune75 exceeds the 500 ms budget despite fitting the 120 KB arena.

fitting the 120 KB arena—accuracy alone does not predict real-time feasibility (Section V). INT8+Prune50 is fastest by a wide margin (3.7× faster than full-width INT8). FP32 additionally requires a 160 KB arena, reducing minimum free heap to 116 KB vs. 156 KB for the 120 KB-arena configs.

V. Discussion

A. Variance Across Folds and Seeds

Across the $K{=}5$ folds$\times$3 seeds ($N{=}15$) baseline runs, $\sigma_{\text{fold}}{=}1.38$ pp exceeds $\sigma_{\text{seed}}{=}0.28$ pp for TCN, so performance spread (Table I) primarily reflects which subjects appear in each test fold rather than initialization noise.

B. Vulnerability of Transition Phases

Across k-fold compression (Table II), PSw F1 shows the largest pruning-related degradation relative to FP32 (-3.75 pp Prune25, -1.59 pp Prune50 5 ep, -1.79 pp INT8+Prune50), while INT8 alone changes it by only $+0.03$ pp ($p{=}0.30$); LR is equally sensitive (-6.98 pp, Prune25), and the fine-tune ablation confirms only partial PSw recovery. This may be confounded with support/difficulty, since LR/PSw are simultaneously the only transition phases and lowest-support classes ($\sim$6%/5%); still, relative to each phase's own FP32 F1 under Prune25, LR/PSw lose more (-8.1%, -4.5%) than steady-state LS/Sw (-1.3%, -1.1%)—not purely a baseline-F1 artifact, though collinear with only four classes. The pattern reappears, more severely, under real deployed INT8 across the full $N{=}15$ protocol (Table III): LR F1 falls 22.1 pp on average vs. 7.4 pp for Sw—transition-phase vulnerability is a general compression signature, not specific to pruning.

C. Role of Dilated Convolutions Under Compression

INT8 weight quantization alone preserved macro F1 within 0.01 pp of FP32 ($p{=}0.30$), so TCN's dilated receptive field and narrow hidden width tolerate 8-bit *weight* precision well. The real deployed model additionally quantizes *activations*, compounding across the four stacked dilated blocks; this alone would predict a roughly uniform gap, but it instead concentrates in fold 4 under every seed (Table III), suggesting specific subjects' activations fall outside the calibration set's covered range. Pruning removes capacity by a different, uniform mechanism (Prune50 0.53 pp below FP32 even at 50-epoch fine-tuning), but PSw suffers under both.

Efficiency-accuracy trade-off: the QDQ proxy overstates deployed INT8 accuracy by 11–13 pp on average (up to 35 pp worst case), and higher accuracy does not imply faster inference: Prune75 has the best no-quantization accuracy (-0.70 pp) yet misses the 500 ms budget (510.8 ms), while Prune50 (-1.47 pp, 253 ms) and INT8+Prune50 (75.7 ms) stay well within it. Both the proxy's accuracy optimism and a pruned config's latency require on-device confirmation before recommending a deployment target from PyTorch-side numbers alone.

D. Limitations

- Results derive from a single public dataset (NONAN GaitPrint; generalization untested); ESP32-C3 benchmarks used a synthetic replay window (fold 0, seed 42) with no live IMU/BLE.
- Test stride 1 yields highly overlapping windows; per-subject blocked aggregation mitigates inflated sample size, but Wilcoxon tests use fold$\times$seed aggregates ($N{=}15$), not fully independent (three seeds share five folds), bounding the smallest signed-rank p at 6.10×10^{-5}; Holm–Bonferroni correction within each table changes no conclusion.
- Prune25/75+INT8 combinations were not part of the compression grid, so no real-TFLite gap was measured for them. Power (mW) was not measured; latency used TFLite Micro reference kernels (Chirale_TensorFlowLite).

VI. Conclusion

This study evaluated how INT8 quantization and structured pruning affect phase-specific accuracy in a Standard TCN for gait phase detection. TCN achieved $90.87\% \pm 1.47\%$ macro F1 under $K{=}5 \times 3$ cross-validation—the highest mean among compared architectures while remaining microcontroller-deployable. Paired Wilcoxon tests ($N{=}15$) showed INT8 statistically equivalent to FP32 ($p{=}0.30$) under a PyTorch weight-only QDQ proxy; however, the *actual deployed* full-integer TFLite model, evaluated on the complete test set across the same $N{=}15$ protocol, showed a far larger loss (mean 11.15 ± 9.51 pp, $p{=}6.1 \times 10^{-5}$), concentrated in one fold under every seed tested (worst case -35.13 pp)—implicating the held-out subject cohort, not calibration randomness. Structured pruning, unaffected by this proxy gap, incurs significant but severity-dependent losses (Prune50 at 5 fine-tune epochs: -1.47 pp; Prune75: -0.70 pp, both $p{<}0.001$), partly recoverable with longer fine-tuning. PSw and LR F1 were the most compression-sensitive transition phases under both pruning and real quantization. Despite its accuracy advantage, Prune75 measured 510.8 ms on-device, missing the 500 ms budget that Prune50 (253 ms) meets comfortably—accuracy alone does not predict deployability. INT8+Prune50 is fastest to deploy (75.7 ms, 13.4 KB) but its real accuracy cost (13.10 ± 6.59 pp) far exceeds its proxy estimate (1.54 pp). Future work will validate live IMU streaming and other cohorts.

References

[1] W. Choi and M.-T. Choi, "Deep learning-based gait phase detection using shank-mounted IMU data: Classification approach," *PLOS ONE*, vol. 21, no. 4, p. e0344002, 2026, doi: 10.1371/journal.pone.0344002.

[2] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[3] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[4] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[5] R. David et al., "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.

[6] Espressif Systems, *ESP32-C3 Technical Reference Manual*, version 1.3, 2024.

[7] T. M. Wiles et al., "NONAN GaitPrint: An IMU gait database of healthy young adults," *Scientific Data*, vol. 10, no. 867, 2023, doi: 10.1038/s41597-023-02704-z.